

La rampa TRX

Encargo completo para el segundo desarrollador: misión, arquitectura, planos, pasos hasta la implementación, cómo se trabaja con este repositorio y qué sigue después.

1. **Parte I** — El proyecto y cómo se trabaja aquí

2. Misión, visión y tu papel

3. Qué es Marea y dónde está hoy

4. Cómo está armado

5. Las reglas que no se rompen

6. **Señales para tu agente**

7. **Parte II** — Tu encargo

8. La decisión que lo define todo

9. Arquitectura de la rampa

10. Los caminos que fallan

11. El contrato de código

12. Los siete pasos

13. Recomendaciones y definición de terminado

14. **Parte III** — Tu agenda

15. Anexos: comandos y glosario

ENCARGO	DURACIÓN ESTIMADA	BLOQUEA A
Rampa USDT-TRC20 → USDC-Base	2–3 semanas · +1–2 la vuelta	nadie – es paralelo
REPOSITORIO	TU RAMA	DOCUMENTO
rasdg05/fq-bot · marea/	claude/marea-rampa-tron	2026-09-07 · v1

Parte I · El proyecto y cómo se trabaja aquí

Léelo entero antes de escribir una línea. Son veinte minutos que ahorran una semana.

01 Misión, visión y tu papel

MISIÓN DE MAREA

Que cualquiera en Latinoamérica pueda opinar sobre un hecho futuro con dinero de verdad, y **comprobar por su cuenta que el resultado y el pago fueron honestos** — sin tener que confiar en nosotros.

Visión. Ser la cámara de compensación de los mercados de predicción en español: la capa aburrida y verificable donde se cruzan las dos partes, con el dinero en un contrato público y la casa cobrando por operar, nunca por ganarle a nadie.

Tu papel. Eres el segundo desarrollador y tu primer encargo es la **puerta de entrada del dinero**. Es el bloque más paralelizable del plan: no toca el contrato, ni la matemática de liquidación, ni ninguna invariante del pozo. Puedes trabajar de principio a fin sin bloquear a nadie y sin que nadie te bloquee.

Pero que sea paralelo no lo hace menor: **es lo primero que toca un usuario real**, y es donde una decisión de arquitectura equivocada puede tirar abajo la propiedad más importante del producto. Eso está en la sección 06 y es lo único de este documento que no se negocia.

02 Qué es Marea y dónde está hoy

App móvil, español latino, donde la gente responde preguntas sobre hechos futuros verificables — inflación, tipo de cambio, precios, Liga MX— y comparte el resultado. Cada mercado cita una fuente pública desde que nace, y un proceso automático la lee, abre una ventana de disputa y paga.

Lo que ya funciona

289 pruebas en verde · ciclo automático de creación, lectura, disputa y pago con nueve fuentes de oráculo · contabilidad de partida doble que cuadra en cero · cuentas con código de recuperación · el compensador puro con su prueba de mil secuencias.

Lo que no existe

Volumen: cero. Se juega con **puntos**, no con dinero. Ningún país habilitado: la tabla arranca entera en «pendiente» y el código se niega a recibir dinero sin opinión legal. Nada de la parte en cadena está desplegado.

LO QUE ESTO SIGNIFICA PARA TU TRABAJO

Vas a construir la rampa contra la puerta cerrada. Con la elegibilidad en «pendiente», `crearSolicitud` lanza excepción — y así debe seguir. Tu trabajo se prueba con el proveedor simulado y con pruebas, no moviendo dinero. Si en algún momento la forma más rápida de avanzar parece ser abrir la puerta «sólo para probar», la respuesta es no.

03 Cómo está armado

marea/	
src/domain/	contratos y reglas puras: pozo, parimutuel, contabilidad, liquidación, elegibilidad, solicitudes ← lo tuyo
src/adapters/	puertos e implementaciones: mock, venues, oráculos, wallet puede/ ← lo tuyo, no existe todavía
src/state/	store único (reducer + acciones) con adapters inyectables
src/screens/	una pantalla por destino + hojas
src/lib/	strings (todo el copy), flags, config
server/*.mts	servidor propio, persistencia JSON atómica
vault/	la documentación viva del producto
tests/	vitest

- 1 UI ≠ datos ≠ wallet ≠ analítica ≠ errores.** Las pantallas sólo hablan con el store; el store sólo habla con los adapters. Si una pantalla importa un adapter, está mal.
- 2 El dominio es puro.** `src/domain/` no hace red, no lee reloj y no tiene estado. Todo lo que entra, entra por parámetro. Es lo que permite probarlo en microsegundos.
- 3 Los adapters son puertos.** Cada integración tiene su interfaz y al menos dos implementaciones: la simulada y la real. La simulada **declara que lo es**.
- 4 Todo el texto vive en `lib/strings.ts`.** No se escriben cadenas visibles dentro de un componente.

04 Las reglas que no se rompen

El repositorio tiene 68 reglas permanentes en `vault/RULINGS.md`. Éstas son las que te van a tocar a ti. Violar una es hallazgo crítico automático, no una discusión de estilo.

- R-022** **Nunca fingir que se habló con un proveedor.** Si la integración no está configurada, la app degrada a su camino simulado y lo dice en la interfaz. Éste es el que más te va a aplicar.
- R-011** **El copy se deriva de las flags,** no de suposiciones. Lo que la app afirma tiene que ser verdad según la configuración real.
- R-045** **La puerta de elegibilidad se aplica al crear,** no al ejecutar. Una solicitud creada en un país sin permiso ya es un camino de dinero abierto, aunque nunca avance.
- R-016** **Un reintento no duplica un movimiento de dinero.** `avanzar()` al estado en que ya se está no hace nada. Tu integración tiene que ser idempotente por la referencia del proveedor.
- R-065** **El pozo es cámara, nunca creador de mercado.** No te toca directamente, pero explica por qué el diseño es como es.
- L12** **Marea no puede mover fondos de un usuario.** No existe la función, y tu encargo es justo donde alguien podría introducirla sin querer. Ver sección 06.

05 Señales para tu agente

Si trabajas con un asistente de código, dale esto. Es la cultura del repositorio comprimida, y es lo que separa un PR que entra de uno que hay que rehacer.

LO PRIMERO QUE TIENE QUE LEER TU AGENTE

Esta lista es un resumen para que puedas leerlo en papel. **La especificación real y autoritativa vive en el repositorio,** y tu agente debe leerla directamente: ``marea/vault/AGENTE.md`` — función objetivo, jerarquía de la verdad, invariantes del producto, presupuestos que no se negocian, protocolo de verificación de seis peldaños y qué no se hace. Si algo de este resumen contradice ese documento, **gana el documento.**

- 01** Antes de tocar nada: ``CLAUDE.md`` en la raíz y ``MEMORY/00-INDICE.md``. Sesenta segundos. El contexto largo vive en ``MEMORY/`` y se lee bajo demanda, no se re-deriva.
- 02** La regla que gobierna el repo: **un hallazgo sin un test que lo haga cumplir es una nota, no un arreglo.** Cuando cierres algo, pregúntate qué prueba impide que vuelva. Si la respuesta es «acordarse», no está cerrado.
- 03** **Prefiere editar a crear.** Este repo ya tiene mucho. Antes de escribir un archivo nuevo, busca si ya existe la pieza. Duplicar es deuda.
- 04** **Antes de proponer algo, mira `MEMORY/CEMENTERIO.md`.** Hay ideas que ya se mataron con razón escrita — incluida la de hacer la rampa con una dirección de depósito nuestra.
- 05** **Sin relojes de pared en las pruebas.** Las fechas y el «ahora» entran por parámetro. Una prueba que depende de la hora a la que corrió es una prueba que va a fallar sola un martes.

- 06 **Una métrica demasiado limpia es un bug**, no un hallazgo. Si tu prueba pasa a la primera y cubre mucho, sospecha de la prueba antes que celebrar el código. Rómpela a propósito y confirma que se pone roja.
- 07 **Comentarios en español y explican por qué**, no qué. El código ya dice qué hace; el comentario dice por qué está así y qué pasa si se cambia.
- 08 **Nunca a `main` directo**. `main` redespiega producción. Trabajas en tu rama y se revisa por PR.
- 09 **`npm run ci` antes de cada commit**: tipos + validación + build. Y `npm run test` para la suite.
- 10 **No inventes números**. Si citas una cifra, cita de dónde sale. Este proyecto ya pagó caro publicar un número que no aguantaba una auditoría.

LAS CUATRO PRUEBAS ROJAS QUE VAS A ENCONTRAR

Al correr la suite vas a ver **4 fallos** en `ownmarkets` y `settlement`. **No son tuyos y no son un bug de código**: el catálogo tiene fechas de julio y agosto y ya caducó. Se arreglan corriendo `npm run roll`, que sale a buscar precios reales y repuebla el catálogo. **Ése es tu primer día** — ver la sección 14.

Parte II · Tu encargo

La rampa: cómo entra el dinero del usuario, sin que pase nunca por nosotros.

06 La decisión que lo define todo

El usuario latinoamericano no llega de un banco: llega con **USDT en Tron**, que es el riel que de verdad usa. Tu encargo es que pueda entrar con eso y terminar con USDC en Base, listo para operar.

Hay dos maneras de construirlo y sólo una es compatible con el proyecto.

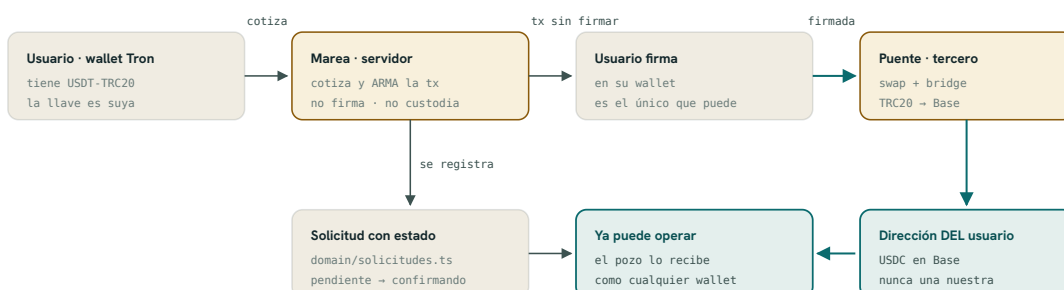
FORMA	QUÉ PASA	VEREDICTO
Integración	El usuario firma. Los fondos van de él → al puente → a su propia dirección en Base. Nunca pasan por nosotros	Ésta
Dirección de depósito	«Manda a esta cuenta y te acreditamos». Nos vuelve custodios en la pata de Tron	Prohibida

La segunda es más fácil de construir, y por eso es la trampa. Un producto no custodial con una rampa custodial es un producto custodial con un diagrama bonito: tira abajo L12, R-065, el argumento legal entero y la propiedad que hace posible pedir una opinión con respuesta. Si en algún momento el camino más corto parece ser «que nos manden a una dirección nuestra», **para y pregunta**. No es una optimización, es un cambio de qué somos.

EL PATRÓN YA EXISTE EN EL REPO — CÓPIALO

Mira ``src/adapters/custodia/contrato.ts``. Es la interfaz del contrato de custodia, y **deliberadamente no expone ``firmar()`` ni ``retirar()``**, con el motivo escrito en el encabezado. Tu ``Puente`` se diseña igual: **no tiene un método que ejecute una transferencia con llaves nuestras**, porque no tenemos ninguna. Cotiza, arma la transacción para que la firme el usuario, y consulta el estado. Nada más.

07 Arquitectura de la rampa



✗ Lo que NO existe en este diagrama, y no se agrega

ninguna dirección nuestra que reciba fondos · ninguna llave nuestra que firme por el usuario
ningún saldo que nosotros «acreditemos» a mano · si aparece cualquiera de las tres, el diseño se rompió

08 Los caminos que fallan

El camino feliz es un día de trabajo. **El resto del encargo son éstos** — y es donde se gana o se pierde la confianza del usuario.

El puente está caído	<code>`cotizar()`</code> falla. La app lo declara (R-022) y deja abierto el camino directo: el que ya tiene USDC en Base entra sin rampa. El pozo y los pagos siguen funcionando.
Llegó menos de lo cotizado	Slippage o comisión del puente. Se acredita lo que llegó, nunca lo cotizado . Y la cotización se le muestra al usuario con su margen antes de que firme.
Se quedó a medias	La solicitud se queda en <code>`confirmando`</code> y no avanza sola . Tiene que seguir siendo consultable días después: el estado vive en el servidor, no en el navegador.
El usuario cerró la app	No pasa nada. Al volver, ve su solicitud en el estado en que esté. Si el estado sólo vivía en el cliente, está mal.
El proveedor reintentó	Idempotencia por la referencia del puente: el mismo aviso tres veces produce un movimiento. <code>`avanzar()`</code> ya es idempotente por estado; tú tienes que serlo por referencia.
Llegó dos veces	Dos transferencias distintas con la misma referencia lógica. Se detecta y se registra; nunca se acredita dos veces.
Confirmaciones insuficientes	Define cuántos bloques antes de dar por buena la llegada, y déjalo configurable , no incrustado en el código.

09 El contrato de código

Propuesta de interfaz. Ajústala si al integrar el proveedor descubres que sobra o falta algo — pero **la ausencia de un método que ejecute con llaves nuestras no se toca**.

```
// src/adapters/puente/contrato.ts
export interface Cotizacion {
  montoOrigen: number;          // USDT que pone el usuario
  montoEstimado: number;        // USDC que estima recibir
  comisionPuede: number;
  margenSlippage: number;       // se le MUESTRA antes de firmar
  expiraEn: string;
  referencia: string;           // la clave de idempotencia
}

export interface TxParaFirmar {
  cadena: "tron";
  destino: string;              // contrato del puente
  datos: string;
  // A dónde llegan los fondos al final. SIEMPRE una dirección
  // del propio usuario. Es lo que fija la invariante L17.
  beneficiarioFinal: string;
}

export type EstadoPuede =
  | { estado: "pendiente" }
  | { estado: "en_transito"; confirmaciones: number }
  | { estado: "completado"; montoRecibido: number; txDestino: string }
  | { estado: "fallido"; motivo: string };

export interface Puente {
  readonly id: string;
  // Si es true, nada de lo que devuelve viene de un proveedor real.
  // Se expone para que la interfaz pueda decirlo (R-022).
  readonly esSimulado: boolean;
  readonly descripcion: string;

  cotizar(i: { montoUsdt: number; beneficiario: string }): Promise<Cotizacion>;
  armar(c: Cotizacion): Promise<TxParaFirmar>;
  estado(referencia: string): Promise<EstadoPuede>;

  // NO existe enviar(), ni firmar(), ni retirar(). No es un olvido:
  // no tenemos llaves de nadie y no vamos a tenerlas (L12).
}
```

LA INVARIANTE QUE TIENES QUE CABLEAR

L17 · el beneficiario final de una transacción de rampa es siempre una dirección del propio usuario. *Test que la fija:* armar una transacción cuyo `beneficiarioFinal` no sea la dirección conectada del usuario **lanza excepción**. Si algún día alguien mete una dirección nuestra ahí, la suite se pone roja antes de que llegue a producción. Ésa es tu contribución más importante a este proyecto, más que el código feliz.

10 Los siete pasos

01 Interfaz y proveedor simulado

2 días

`adapters/puente/contrato.ts` con la interfaz, y `puente/simulado.ts` que la implementa declarando `esSimulado: true`. Cablearlo en `adapters/index.ts` con el patrón que ya usan los demás: sin configuración, cae en el simulado.

Puerta: `npm run ci` verde y la app dice en pantalla que la rampa es simulada. Nunca finge.

02 La máquina de estados, enchufada

2 días

Conectar con `domain/solicitudes.ts`, que ya existe y ya tiene las transiciones legales. Añadir la referencia del puente a la solicitud. **No reescribas la máquina:** se extiende.

Puerta: test de idempotencia — el mismo aviso del proveedor tres veces produce un solo movimiento.

03 Cotizar y armar

3 días

La cotización con su margen visible, y el armado de la transacción sin firmar. Aquí vive L17.

Puerta: el test de L17 en rojo si el beneficiario no es del usuario. Rómpelo a propósito una vez y confirma que se pone rojo.

04 Monitoreo del estado

3 días

Consulta periódica del estado por referencia, avance de la solicitud, y persistencia en el servidor.

Puerta: con el puente sin responder, la solicitud queda consultable — no perdida y no acreditada.

05 Montos parciales y slippage

2 días

Acreditar lo recibido, no lo cotizado. Registrar la diferencia.

Puerta: test de «llegó menos» — se acredita el monto real y el usuario ve por qué difiere.

06 La pantalla

3 días

Estado del depósito, con texto en `lib/strings.ts`. Cada estado dice la verdad, incluida la caída del proveedor.

Puerta: los siete caminos de la sección 08 tienen su texto, y ninguno miente.

07 Proveedor real, endurecimiento y documentación

3–4 días

La implementación contra el puente elegido, detrás de configuración. Y un documento en `vault/` que explique cómo operarla.

Puerta: otra persona puede operar la rampa leyendo tu documento, sin preguntarte.

11 Recomendaciones de implementación

- 01 **Elige el puente al final, no al principio.** Construye contra tu interfaz con el simulado. Cuando llegues al paso 07 vas a saber qué necesitas de verdad, y la comparación entre proveedores será de treinta minutos en vez de una semana de prueba y error.
- 02 **Los criterios para elegirlo,** en este orden: que soporte TRC20 → Base con USDC nativo · que la transacción la firme el usuario y no exija custodia · que exponga consulta de estado por referencia · comisión y slippage declarados antes de firmar · y qué te exige a ti (varios piden verificación de la empresa: **eso es cola, avísalo pronto**).
- 03 **No confíes en el cliente.** El monto, el beneficiario y el estado se verifican en el servidor contra el proveedor. Lo que manda el navegador es una intención, no un hecho.
- 04 **Idempotencia por referencia, desde el primer commit.** Meterla después es reescribir. La referencia del puente es la clave, no un id nuestro.
- 05 **Degrada declarando.** Sin proveedor configurado, la rampa no desaparece de la interfaz: aparece diciendo que está simulada. Es la diferencia entre un producto honesto y uno que aparenta.
- 06 **Prueba sin dinero.** Todo el encargo se puede completar con el simulado y con pruebas. Cuando toque probar contra el proveedor real, hazlo en su red de prueba y con montos mínimos, y coordínalo antes.
- 07 **Escribe el texto de los errores tú.** «Error 502» no es un mensaje. «El puente no responde; tu dinero no se movió y puedes intentar en unos minutos» sí. Y va en `lib/strings.ts`.

DEFINICIÓN DE TERMINADO

El encargo está terminado cuando: **(1)** los siete caminos de fallo tienen prueba y texto · **(2)** el test de L17 existe y se pone rojo si alguien mete otra dirección · **(3)** la suite completa pasa sin tocar expectativas de pruebas ajenas · **(4)** `npm run ci` verde · **(5)** hay un documento en `vault/` con el que otra persona opera la rampa · y **(6)** con el proveedor sin configurar, la app lo declara y sigue usable.

Parte III • Tu agenda

Qué haces el primer día, y qué sigue cuando la rampa esté cerrada.

12 Día 1 — antes de la rampa

Una tarea pequeña y real para entrar al repositorio con las manos, no leyendo:

D1

Repoblar el catálogo y poner la suite entera en verde

medio día

Hay 4 pruebas rojas porque el catálogo tiene fechas caducadas. Corre ``npm run roll``, verifica que el feed vuelve a tener mercados vigentes, y deja la suite en 293 verdes.

Por qué es buen primer día: tocas el ciclo automático, ves cómo se generan y resuelven los mercados, y entiendes por qué existe la regla de que un catálogo con fechas fijas caduca — sin arriesgar nada.

13 Después de la rampa, en orden

#	TRABAJO	POR QUÉ EN ESE ORDEN	EST.
1	La pata de vuelta — retirar de Base a Tron	Es la mitad que falta de tu propio encargo. Más delicada que la entrada: aquí el usuario saca	1-2 sem
2	Screening de sanciones	No es KYC y no admite niveles: aplica a todos. Son 2-3 días y desbloquea el lanzamiento. Se coordina con el brazo legal	3 días
3	Verificador de prueba — pantalla donde el usuario comprueba su operación contra la raíz anclada	Independiente, no toca dinero, y es la cara visible de la auditabilidad	1 sem
4	Vigilante externo — proceso que compara el colateral del contrato contra los conjuntos emitidos, cada bloque	Es una invariante que hoy es sólo un test; falta como servicio que grite	1 sem
5	Niveles de verificación N0-N3 con tope efectivo	La estructura se puede construir ya; el número del tope espera la respuesta legal	1-2 sem

Los puntos 1 y 2 son los que están en el camino crítico del lanzamiento. Del 3 al 5 se pueden reordenar según lo que haga falta.

14 Cómo trabajamos

Ramas Una por encargo, con nombre descriptivo. Nunca a ``main``: ``main`` redespiega producción.

PRs Pequeños y con el porqué en la descripción. Un PR que sólo dice qué hace obliga a leer el diff para entenderlo.

Decisiones Lo que cambia una regla, una invariante o la forma del dinero **se pregunta**. Lo que es implementación dentro de tu encargo, lo decides tú. Ante la duda, la sección 06 es el ejemplo de lo que se pregunta.

Memoria Cuando cierres algo que costó descubrir, déjalo en `MEMORY/marea/` con fecha. La memoria es lo que evita que el proyecto vuelva a discutir lo mismo en seis meses.

A Anexo • Comandos

npm install	instalar
npm run dev	desarrollo
npm run test	suite (vitest)
npm run ci	tipos + validación + build ← antes de cada commit
npm run validate	el reporte de validación del producto
npm run roll	repuebla el catálogo con los mercados de la semana
npm run settle	liquida los mercados que ya resolvieron
npm run daily	las tres tareas de mantenimiento, en orden

B Anexo • Glosario

cámara	La pieza que guarda el colateral de las dos partes y paga al que acertó. No toma lado nunca.
conjunto completo	Una unidad de colateral equivale a un contrato de cada resultado. Es lo que hace que el pozo no tenga varianza.
taker / maker	Quien toma un precio que ya estaba / quien lo pone. Hoy no hay libro; es vocabulario del diseño futuro.
subsidio	Liquidez que pone la casa para que un mercado nuevo no esté vacío. Tiene tope y nunca cobra .
época / ancla	El libro se encadena con hashes y cada cierto tiempo su raíz se publica en la cadena. Eso permite a un usuario verificar su operación sin ver la de nadie.
oráculo	El proceso que lee la fuente pública citada por el mercado y publica el resultado con su evidencia.
ventana de disputa	El tiempo entre que se lee el resultado y se paga. No se paga con la ventana abierta, aunque el resultado se sepa.
elegibilidad	La tabla por país que decide si se puede depositar y operar. Hoy está entera en «pendiente» y el código lo hace cumplir.